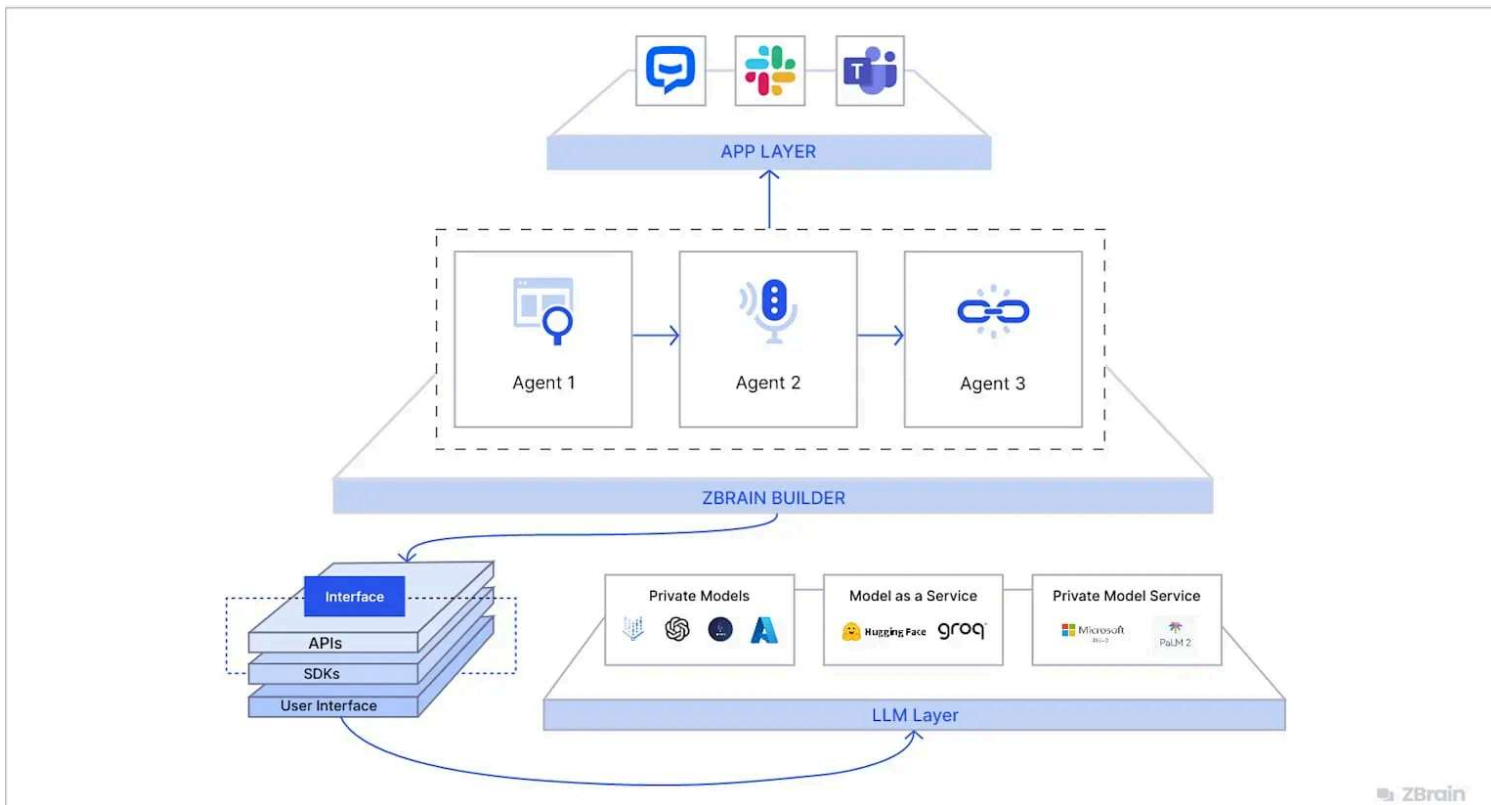


T3CH



<https://zbrain.ai/how-zbrain-multi-agent-systems-work/>

This member-only story is on us. [Upgrade](#) to access all of Medium.

★ Member-only story

OpenClaw Multi-Agent Deployment: From Single Agent to Team Architecture — The Complete Path

AI Agent



JIN

Follow

12 min read · Mar 8, 2026

85



Disclosure: I use GPT search to collection facts. The entire article is drafted by me.

You've been running OpenClaw for a few weeks. The single-agent setup works fine for basic tasks. But then something shifts.

Your writing agent starts mixing technical documentation context with casual chat history. Your development agent surfaces personal reminders during code reviews. Memory leaks across contexts. System prompts balloon to 40,000 tokens because one agent is trying to be everything at once.

That's the signal. Time to architect a team.

This isn't about academic multi-agent frameworks where agents delegate through natural language. We're talking about a practical production pattern: multiple isolated agents, each with distinct memory, workspace, and responsibility, all running on your infrastructure.

As of March 2026, OpenClaw's latest release (2026.2.17) introduced deterministic sub-agent spawning and structured inter-agent communication. The community has deployed everything from personal three-agent setups to enterprise multi-tenant architectures. Security research revealed both the power and the risks — token exhaustion attacks demonstrated 6–7x amplification through tool chains, while session isolation failures led to CVE-level vulnerabilities.

The engineering path from single to multi-agent isn't just adding config lines. It's rethinking routing, isolation, and state management. This guide maps the entire journey.

What One Agent Actually Includes

Before splitting agents, understand what you're replicating.

An OpenClaw agent isn't just a name or system prompt. It's a completely isolated execution unit consisting of three independent resource layers:

```
// ~/.openclaw/openclaw.json
{
  "agents": {
    "list": [{
      "id": "main",
      "workspace": "~/.openclaw/workspace-main",
      // Three distinct resource boundaries:
      // 1. Workspace: personality + memory + files
      // 2. AgentDir: auth + model config + state
      // 3. Sessions: conversation JSONL storage
    }]
  }
}
```

Resource Layer Breakdown:

Resource	Purpose	Default Path	Isolation
Workspace	Personality files (SOUL.md, AGENTS.md, USER.md), memory, working files	~/.openclaw/workspace-<agentId>	Per-agent, tool execution cwd
AgentDir	Authentication tokens, model configs, state persistence	~/.openclaw/agents/<agentId>/	Per-agent credentials
Sessions	Conversation history (JSONL format)	~/.openclaw/agents/<agentId>/sessions/	Isolated by session scope

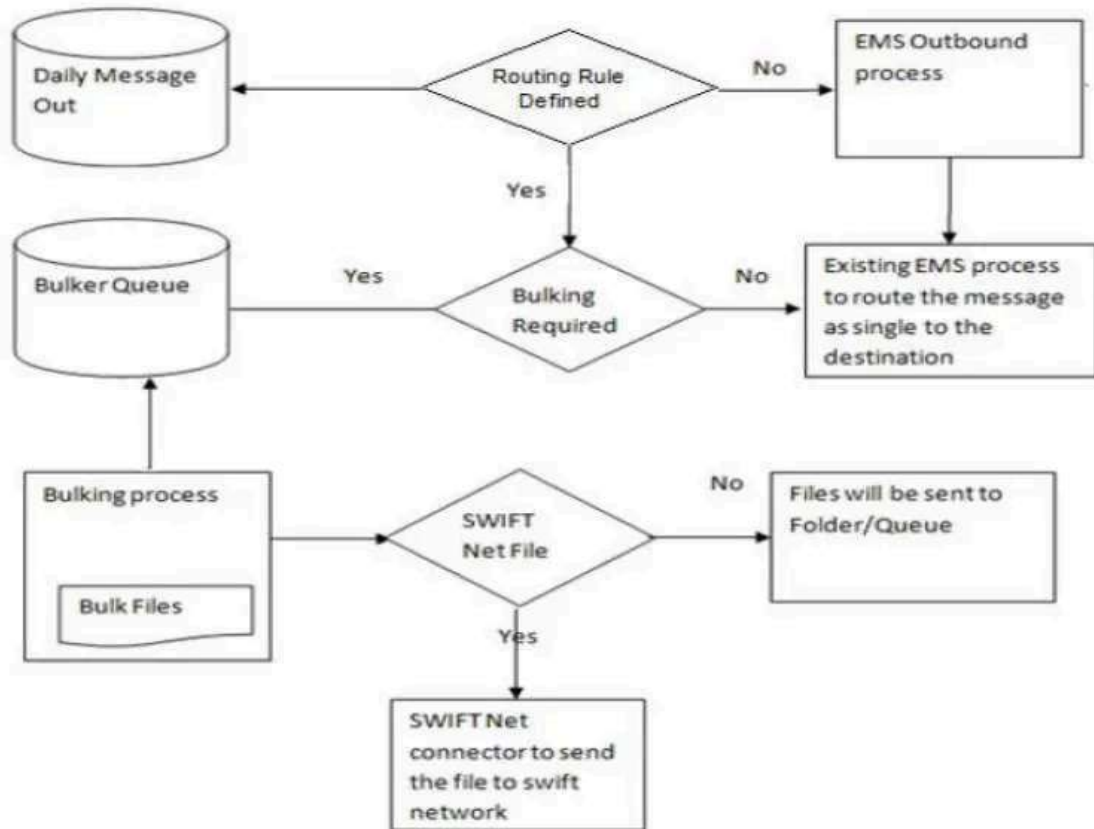
The workspace is where tools execute — all relative paths resolve from there. AgentDir stores credentials that never cross agent boundaries. Sessions persist conversation state with configurable compaction policies.

Critical detail: **Workspace isolation isn't hard isolation.** Tools can still access absolute paths on the host. For true containment, you need Docker sandboxes (covered later).

When you see context contamination, you're often seeing shared workspace pollution. When system prompts explode, one agent is accumulating rules meant for different roles. When authentication breaks unexpectedly, agents are stepping on each other's AgentDir state.

Multi-agent architecture fixes this by giving each agent its own universe.

The Message Flow: Single vs. Multi-Agent



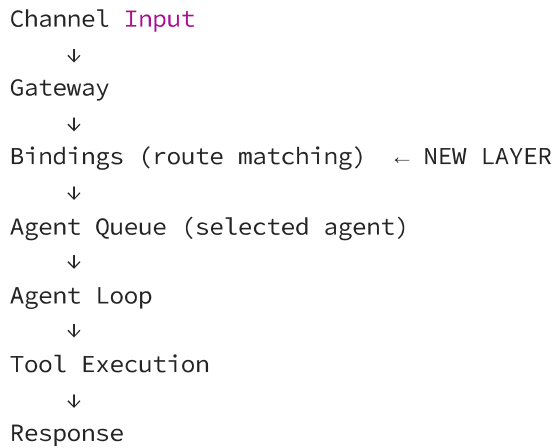
<https://docs.oracle.com/en/industries/financial-services/banking-payments/14.8.0.0.0/obpmg/message-flow-diagrams.html>

In single-agent mode, the flow is linear:

```
Channel Input
↓
Gateway (WebSocket server)
↓
Command Queue
↓
Agent Loop (context assembly + model invocation)
↓
Tool Execution
↓
Response Delivery
```

One channel, one agent, one execution path. Simple. Deterministic.

Multi-agent introduces a **routing layer** before the agent loop:



The Gateway receives a message, evaluates `bindings` rules, determines which agent should handle it, then dispatches to that agent's queue. Each agent maintains a separate state, separate model configs, and separate memory.

This routing logic is **deterministic**. No LLM decides where messages go. You define explicit rules: “WhatsApp work account messages go to `work` agent, Telegram DMs go to `personal` agent, this specific group chat goes to `support` agent.”

The Gateway becomes the orchestration control plane. Agents become specialized workers.

When to Split: Three Red Flags

Don’t prematurely architect. Start with one agent. Split when you see these signals:

1. Memory Cross-Contamination

You ask the writing agent about your latest article draft. It responds with details from yesterday’s work Slack thread.

Root cause: Shared workspace means shared `USER.md`, shared memory files, shared conversation context. Session compaction merges unrelated history

into the same summary.

Cost: Privacy leaks, inaccurate responses, loss of context boundaries.

2. System Prompt Bloat

Your agent's system prompt hits 35,000 tokens because it's simultaneously:

- A Python developer (coding standards, linting rules, test frameworks)
- A content writer (tone guidelines, SEO rules, publication workflows)
- A personal assistant (calendar preferences, communication style, relationship context)

Root cause: One agent trying to be everything.

Cost: Higher API bills, slower inference, context window pressure, and degraded instruction following.

3. Compaction Blocking

Session compaction runs every N messages to keep context manageable. With one agent handling all conversations, compaction on a long support thread blocks compaction for your active coding session.

Root cause: Shared session store means serialized compaction.

Cost: Latency spikes, context freshness issues, race conditions under load.

When any of these appear, multi-agent isn't premature optimization — it's an architectural necessity.

Creating Multiple Agents: CLI and Config

OpenClaw provides a simple CLI for agent creation:

```
# Create three specialized agents
openclaw agents add writing
openclaw agents add development
openclaw agents add personal
```

Each command automatically generates:

- Workspace directory with default personality files
- AgentDir with isolated state
- Session storage directory
- Configuration entry in `openclaw.json`

Manual configuration gives more control:

```
// ~/.openclaw/openclaw.json
{
  "agents": {
    "list": [
      {
        "id": "writing",
        "default": false,
        "workspace": "~/.openclaw/workspace-writing",
        "model": "anthropic/claude-sonnet-4", // Optimized for prose
        "skills": ["research", "markdown", "seo"]
      },
      {
        "id": "development",
        "default": false,
        "workspace": "~/.openclaw/workspace-dev",
        "model": "anthropic/claude-opus-4", // Reasoning-heavy
        "skills": ["code", "git", "docker", "testing"]
      },
      {
        "id": "personal",
        "default": true, // Handles unmatched messages
        "workspace": "~/.openclaw/workspace-personal",
        "model": "anthropic/claude-sonnet-4"
      }
    ]
  }
}
```

Critical rules:

- **Each agent MUST have a unique workspace.** Shared workspaces defeat isolation.
- **AgentDir paths cannot overlap.** Authentication tokens must stay agent-scoped.
- **One agent marked `default: true`.** This catches unrouted messages.

Verify your setup:

```
openclaw agents list --bindings
```

This shows all agents and their current routing rules. If bindings are empty, all messages hit the default agent.

Message Routing: The Bindings System

Routing is where multi-agent architecture lives or dies.

Bindings define **match rules** that map incoming messages to agents. The Gateway evaluates rules in order, selects the first match, and dispatches to that agent.

Basic Channel Routing

Route by messaging platform:

```
{
  "bindings": [
    {
      "agentId": "writing",
      "match": { "channel": "telegram" }
    },
    {
      "agentId": "development",
      "match": { "channel": "slack", "teamId": "T1234ABCD" }
    },
    {
      "agentId": "personal",
      "match": { "channel": "whatsapp" }
    }
  ]
}
```

This is clean but coarse-grained. All Telegram messages go to `writing`, regardless of who sent them or which chat.

Peer-Level Routing

For finer control, match specific users or groups:


```

{
  "bindings": [
    {
      "agentId": "development",
      "match": {
        "channel": "telegram",
        "peer": {
          "kind": "group",
          "id": "-1001234567890" // Specific group ID
        }
      }
    },
    {
      "agentId": "personal",
      "match": {
        "channel": "telegram",
        "peer": {
          "kind": "direct",
          "id": "+15551230001" // Specific user phone
        }
      }
    },
    {
      "agentId": "writing",
      "match": { "channel": "telegram" } // Fallback for other Telegram
    }
  ]
}

```

Match priority (most specific to least):

1. peer (specific group/DM)
2. parentPeer (thread parent)
3. guildId + roles (Discord server + role)
4. guildId (Discord server)
5. teamId (Slack workspace)
6. accountId (specific account on multi-account channel)
7. accountId: "*" (any account)
8. default agent (catches everything else)

OpenClaw evaluates top-to-bottom, first match wins. Always put specific rules before general fallbacks:

```
// ✅ CORRECT: Specific before general
[
  { match: { channel: "slack", peer: { kind: "direct", id: "U12345" } }, agentId: "support" },
  { match: { channel: "slack", teamId: "T12345" }, agentId: "work" },
  { match: { channel: "slack" }, agentId: "support" }
]

// ❌ WRONG: General rule shadows specific
[
  { match: { channel: "slack" }, agentId: "support" }, // This catches everything
  { match: { channel: "slack", teamId: "T12345" }, agentId: "work" } // Never reached
]
```

Multi-Account Routing

For channels supporting multiple accounts (WhatsApp, Telegram), route by account:

```
{
  "bindings": [
    {
      "agentId": "work",
      "match": {
        "channel": "whatsapp",
        "accountId": "business-line" // Work phone number
      }
    },
    {
      "agentId": "personal",
      "match": {
        "channel": "whatsapp",
        "accountId": "personal-line" // Personal phone
      }
    }
  ]
}
```

One WhatsApp channel, two phone numbers, two agents. Clean separation.

Session Security: The dmScope Configuration

If your agent receives DMs from multiple people, **default session scoping is dangerous.**

By default, `dmScope = "main"`. This means:

- Alice DMs the agent: “I’m feeling anxious about the medical test results.”

- Bob DMs the agent: “What were we discussing earlier?”
- The agent responds to Bob with Alice’s medical context.

Catastrophic privacy leak.

The fix: configure proper session isolation:

```
{
  "session": {
    "dmScope": "per-channel-peer" // Isolate by channel + sender
  }
}
```

dmScope Value	Isolation Level	Use Case
main	All DMs share one session	Dangerous for multi-user
per-peer	Isolate by sender (cross-channel continuity)	Single-user, multiple devices
per-channel-peer	Isolate by channel + sender	Recommended for multi-user
per-account-channel-peer	Maximum isolation	Multi-tenant, high-security

Session isolation failures have led to documented CVEs. Security researchers demonstrated context leakage where admin sessions bleed into standard user contexts under concurrency. OpenClaw’s community openly discusses these risks.

For any production deployment with more than one user, **use** `per-channel-peer` **minimum**.

Three Isolation Strategies

Multi-agent systems can mean different things depending on your isolation requirements.

Strategy 1: Soft Isolation (Shared Process)

All agents run in one Gateway process.

Architecture:

- Single `openclaw` gateway process
- Multiple agents defined in config
- Workspace separation only
- Shared memory space

Configuration:

```
{
  "agents": {
    "list": [
      { "id": "agent1", "workspace": "~/openclaw/workspace-1" },
      { "id": "agent2", "workspace": "~/openclaw/workspace-2" },
      { "id": "agent3", "workspace": "~/openclaw/workspace-3" }
    ]
  }
}
```

Pros: Lowest overhead, simple setup, easy inter-agent communication.

Cons: One crash kills all agents. No kernel-level isolation. Shared process memory means potential side-channel leaks.

Best for: Personal use, trusted contexts, development environments.

Strategy 2: Docker Sandbox (Container Isolation)

Gateway runs on a host, and tool execution happens in containers.

Architecture:

- Gateway process on the host
- Each agent's tool calls execute in ephemeral Docker containers
- Workspace mounted read-only or not at all
- Network isolation configurable

Configuration:

```

{
  "sandbox": {
    "mode": "non-main", // Sandbox all except main agent
    "scope": "session", // One container per session
    "workspaceAccess": "none", // No host workspace access
    "network": "none" // No internet from sandbox
  }
}

```

Sandbox scope options:

Scope	Container Lifecycle	Use Case
global	One container for all sessions	Lowest overhead
session	One container per conversation	Balance of isolation + performance
tool-call	New container per tool invocation	Maximum isolation, highest overhead

Before enabling, build the sandbox image:

```

cd openclaw
./scripts/sandbox-setup.sh

```

Pros: Strong isolation without multiple Gateway processes. Tool execution can't touch the host filesystem or network (if configured).

Cons: Docker overhead. Requires image maintenance. The host filesystem is still accessible unless `workspaceAccess: "none"`.

Best for: Running untrusted skills, production deployments, security-conscious personal use.

Strategy 3: Multi-Gateway (Process Isolation)

Run separate Gateway processes, each with an independent state.

Architecture:

- Multiple `openclaw gateway` processes

- Each Gateway has an isolated config, state, workspace, and port
- Separate model credentials per Gateway
- Full OS-level process isolation

Set up using profiles:

```
# Terminal 1: Main Gateway
openclaw --profile main gateway

# Terminal 2: Rescue Gateway (high-priority fallback)
openclaw --profile rescue gateway

# Terminal 3: Personal Gateway
openclaw --profile personal gateway
```

Each profile points to separate directories:

```
~/.openclaw/profiles/
├── main/
│   ├── openclaw.json
│   ├── state/
│   └── workspace/
├── rescue/
│   ├── openclaw.json
│   ├── state/
│   └── workspace/
└── personal/
    ├── openclaw.json
    ├── state/
    └── workspace/
```

Port assignment: Each Gateway needs a unique port. Recommended spacing: ≥ 20 apart (e.g., 18789, 18809, 18829).

Pros: Maximum isolation. One Gateway crash doesn't affect others. True multi-tenancy. Independent scaling.

Cons: Highest resource usage. Complex inter-Gateway communication. Operational overhead.

Best for: Multi-tenant SaaS, high-availability systems, large teams with separate security domains.

Deterministic Sub-Agent Spawning (2026.2.17+)

The February 2026 release introduced **structured sub-agent creation**.

Instead of letting the main agent decide when to spawn helpers, you trigger sub-agents explicitly:

```
# In chat with main agent:
/spawn reviewer --context "Review the code in ./src/api.ts"
```

This creates a deterministic sub-agent with:

- Its own session
- Configurable tool permissions
- Direct messaging capability to the parent

Inter-agent communication uses the `agentToAgent` tool:

```
{
  "tools": {
    "agentToAgent": {
      "enabled": true,
      "allow": ["programmer", "reviewer", "tester"] // Whitelist
    }
  }
}
```

Agents can now send structured messages to each other:

```
// Programmer agent sends code to reviewer agent
await agentToAgent({
  targetAgent: "reviewer",
  message: "Please review this implementation",
  context: { filePath: "./src/api.ts", changes: "..." }
});
```

This enables **deterministic pipelines** without natural language orchestration overhead. One community member built a code → review → test pipeline where each agent has strict tool boundaries: the programmer gets `exec` and `write`, reviewer gets `read only`, tester gets `exec` + test runners.

Nested spawning is supported up to a configurable depth. A research agent can spawn a fact-checker agent, which can spawn a citation-validator agent.

Remote Access and Security

Gateway binds to `127.0.0.1` by default. **Never expose Gateway ports directly to the internet.**

Recommended access patterns:

1. SSH tunnel (simplest):

```
# On local machine, tunnel to remote server
ssh -L 18789:127.0.0.1:18789 user@server.example.com

# Access Gateway at localhost:18789
```

2. VPN (best for always-on access):

- Use Tailscale, WireGuard, or similar

[Learn more about VPN access](#) Open in app ↗



Medium



Search



Write



3. Reverse proxy with mTLS (enterprise):

- Nginx/Caddy with client certificates
- Gateway behind an authenticated proxy
- Rate limiting and DDoS protection

What NOT to do:


```
// ❌ DANGEROUS: Binding to public interface
{
  "gateway": {
    "host": "0.0.0.0", // Exposes to internet
    "port": 18789
  }
}
```

This makes your agent publicly accessible with no authentication. Attackers demonstrated token exhaustion attacks, achieving 6–7x cost amplification through tool-calling chains. Without proper isolation, malicious inputs can execute arbitrary code.

Configuration Hot Reload

OpenClaw supports live config updates.

Default reload mode: hybrid

```
{
  "reload": {
    "mode": "hybrid" // Hot-reload when possible, restart otherwise
  }
}
```

Reload behavior:

Change Type	Action	Downtime
Bindings update	Hot reload	None
Agent workspace change	Hot reload	None
Model config	Hot reload	None
Channel authentication	Restart required	Brief
Sandbox settings	Restart required	Brief

After editing `openclaw.json`, the Gateway detects changes and reloads automatically. No manual restart for most configuration updates.

For critical changes (authentication, sandbox), Gateway restarts gracefully — completes in-flight requests before restarting.

Architecture Selection Guide

Choose based on actual requirements, not a hypothetical scale.

Scenario	Recommended Approach	Why
Personal use, single user	Single agent	Simplicity. No isolation needed.
Personal, memory mixing	Multi-agent (soft isolation)	Clean separation without overhead.
Small team (2-5 people)	Multi-agent + <code>per-channel-peer</code> <code>dmScope</code>	Session isolation prevents leaks.
Untrusted skills/tools	Docker sandbox	Container isolation limits damage.
Sensitive data handling	Docker sandbox + network isolation	Defense in depth.
Multi-tenant SaaS	Multi-Gateway	Full isolation per tenant.
High availability	Multi-Gateway with rescue profile	Fault tolerance through redundancy.

Evolution path:

Single Agent

↓ (memory contamination)

Multi-Agent (soft isolation)

↓ (security requirements)

Docker Sandbox

↓ (scale/tenant isolation)

Multi-Gateway

Each step adds complexity. Only advance when the current architecture fails your actual use case.

Production Deployment Example

Here’s a real-world configuration for a small business running on DigitalOcean App Platform:

```
// ~/.openclaw/openclaw.json
{
  "agents": {
    "list": [
      {
```

```

    "id": "sales",
    "workspace": "~/.openclaw/workspace-sales",
    "model": "anthropic/claude-sonnet-4",
    "skills": ["crm", "email", "calendar"]
  },
  {
    "id": "support",
    "workspace": "~/.openclaw/workspace-support",
    "model": "anthropic/claude-sonnet-4",
    "skills": ["docs", "troubleshooting", "ticketing"]
  },
  {
    "id": "internal",
    "default": true,
    "workspace": "~/.openclaw/workspace-internal",
    "model": "anthropic/claude-opus-4",
    "skills": ["code", "analytics", "reporting"]
  }
]
},

"bindings": [
  {
    "agentId": "sales",
    "match": {
      "channel": "slack",
      "guildId": "C12345",
      "peer": { "kind": "group", "id": "sales-team" }
    }
  },
  {
    "agentId": "support",
    "match": { "channel": "whatsapp", "accountId": "support-line" }
  },
  {
    "agentId": "internal",
    "match": { "channel": "telegram" }
  }
],

"session": {
  "dmScope": "per-channel-peer" // Multi-user safety
},

"sandbox": {
  "mode": "non-main", // Sandbox sales + support, trust internal
  "scope": "session",
  "workspaceAccess": "read-only",
  "network": "none"
}
}

```

This configuration:

- Routes sales Slack to `sales` agent with CRM tools
- Routes support WhatsApp to `support` agent with docs access
- Routes internal Telegram to `internal` agent with full privileges

- Isolates all DM sessions
- Sandboxes external-facing agents
- Trusts internal agent (likely used by team, not customers)

DigitalOcean App Platform provides elastic scaling — add capacity to individual agents without re-architecting.

Conclusion: Progressive Complexity, Demand-Driven

Multi-agent architecture isn't "better." It's **necessary when single-agent constraints become pain points.**

The path:

1. **Start simple.** One agent, one workspace. Get it running.
2. **Split when memory is contaminated.** Add agents for distinct roles.
3. **Isolate when security matters.** Add Docker sandbox.
4. **Separate when scale demands.** Add a multi-Gateway.

Each step solves a real problem:

- Memory mixing → Multi-agent with separate workspaces
- Privacy leaks → Session isolation via `dmScope`
- Security risks → Docker sandbox
- Tenant isolation → Multi-Gateway

Don't skip steps. Don't prematurely architect. **Build for current needs, refactor when those needs change.**

As of March 2026, OpenClaw's multi-agent capabilities are production-ready. The framework has moved from "local AI assistant" to "agent operating system". Community deployments span from single Mac Minis running three-agent teams to elastic cloud platforms scaling dozens of specialized agents.

The architecture exists. The tooling works. The question is: what team structure does your use case actually need?

Start there. Build incrementally. Let demand drive complexity.

That’s the only sustainable path for agent systems.

If you’d like to show your appreciation, you can support me through:

🌟 [Patreon](#)

🌟 [Ko-fi](#)

🌟 [BuyMeACoffee](#)

Every contribution, big or small, fuels my creativity and means the world to me. Thank you for being a part of this journey!

AI Agent

Artificial Intelligence

Software Development

Software

Generative Ai Tools



Published in T3CH

1.93K followers · Last published 8 hours ago

Follow

Snoop & Learn about Technology, AI, Hacking, Coding, Software, News, Tools, Leaks, Bug Bounty, OSINT & Cybersecurity !!! But, not limited 2, anything that is Tech Linked...You’ll probably find here ! ;)—Stay ahead with Latest Tech News! -> You write about? Just ping to join !



Written by JIN

7.5K followers · 324 following

Follow



https://substack.com/@jinlow?utm_source=user-menu

No responses yet



Jackson

What are your thoughts?

